

Conventions:

SS: Storage server

NS/NM: Naming server

User-client: user/client/user-client, used interchangeably.

Only POSIX C-libraries are allowed to be used.

General:

1. **Ns instances:** There is one instance of ns. Ns down -> system down.
2. **Multiple-client:** Ns should be able to handle multiple clients connecting to it. The clients may request different operations. Ns must handle these operations. Ns must maintain a list of all the currently connected clients (current users list), and also persistently maintain a list of all the past connected clients (all users list). When a client logs off, their name is removed from the current users list.
3. **Client-init:** When a client connects, it gives the ns its username, for file access purposes. Only this simple validation is there, there are no passwords or anything. The ns just receives the name that the client has given it and accepts that it is indeed the name of the client. If client-2 gives the same username as some other currently active client-1 of the same username, it should tell client-2 to join with some other name. All further access type verification after a client joins is done by ns using the username provided by the client.
4. **Multiple-ss:** Ns should be able to handle multiple ss connecting to it.
5. **Ss directory structure:** Ss stores and serves some requests related to files. The directory structure in ss is as follows. All of these are persistent.:
 - a. **File_dir:** contains all the files created by Clients. Clients can only see this directory's contents.
 - b. **Undo_dir:** For every file in file_dir, undo_dir has a version of the previous state of the file. The undo file has the same name as the file in file_dir. So if I opened a file and wrote to it, the previous commit was an empty file with no content. Only one previous copy needs to be stored. So only one undo operation on a file until there is a new change to the file. For eg, file_dir/file1.txt has an undo file undo_dir/file1.txt. file_dir/newdir/file2.txt has an undo file undo_dir/new_dir/file2.txt.
 - c. **Checkpoint_dir:** For every file in file_dir, checkpoint_dir contains atmost 2 checkpoints: <filename>_1 and <filename>_2. There is no ordering between the checkpoints, i.e. it is not necessary that checkpoint 1 has a state of file which is in the past compared to checkpoint_2. Its just that the user will specify with 1 or 2 which checkpoint must be created. When a user wants to save the current state of the file as a checkpoint, the corresponding checkpoint file is created. For example, file_dir/file.txt can have checkpoints file_dir/file.txt_1 and file_dir/file.txt_2.

- d. **Info_dir:** contains persistently some of the information regarding each file, for example, the username of owner of the file, the file creation time, file last updated time, size of file, word count of file, list of users with read access to file, list of users with write access to file, list of users with execute access to the file etc. Again, it follows the same directory structure as file_dir. Eg. file_dir/file.txt has its info file at info_dir/file.txt and file_dir/newdir/file.txt has its info file at info_dir/newdir/file.txt.
6. **Ns file indexing:** Every time an ss connects to the ns, it must send all the files inside it (in the directory called file_dir) to the ns. This includes path of the file relative to file_dir. So file_dir/lol.txt should be sent as lol.txt and file_dir/another_dir/file.txt must be sent as another_dir/file.txt. The ss should also send all the files inside info_dir to the ns.
7. **Ns file indexing particulars:** The ns indexes the files. All the files from all ss should be in the same list and they should not show as per-ss. So ss-1 has lol.txt and ss-2 has file.txt so the ns indexing should have file.txt and lol.txt. The file indexing done by the ns must be efficient (should use an efficient data structure like some hashing structure/tree.)
 For each file, ns should store information of all ss that have given a file with that name. If a file is there in multiple ss, multiple of them will give that file name, as well as its info file. So the ns should also store info like, “what all ss is this file on?”
8. **Same file, multiple ss:** When there are multiple ss giving the same file name, the ns doesn't need to handle complicated stuff like “which storage server is this client requesting his file from.” The ns simply stores the file names, and the first info_file for the file name that it gets, and info of all the ss that have given it the file with that name. If a client requests a file, any of the ss can be chosen to serve it. If ss-1 and ss-3 had different instances of lol.txt, we do not care. Only one lol.txt is sufficient.
 Eg. suppose ss-1 and ss-2 both had lol.txt. Assume ss-1 connected before ss-2. Then ns stores both of them as the ss from which ‘lol.txt’ can be served, and stores the info file of the lol.txt from ss-1 (because ss-1 came first). Note that if ss-2 had lol.txt in file_dir/newdir/lol.txt, then files given by ss-1 and ss-2 would have been different files, because their paths are lol.txt as given by ss-1 and newdir/lol.txt as given by ss-2. So in this case the files are not the same. The ns needs to store the path that is given by an ss, it should not strip the leading directories given by the ss.
9. **Heartbeat:** Ns MUST determine whether a storage server is active, or has gone down. For this purpose, there is a heartbeat that it sends to all ss, every few seconds. The ss must acknowledge this heartbeat and send back a packet acknowledging the heartbeat packet it has received. If an ss is down, ns will not receive any acknowledgement from the ss about this packet. So it must note that the ss is down, and from the file indexing, it must remove any instances of that ss.
10. **The file:**
 - a. The file is a simple text file of only ascii character which is stored on ss. Ss can read this file and send to some client, or modify this file by writing to it. A file can be read from/written to concurrently. So proper locking should be handled. More info on different types of file operations is explained further.

- b. Whenever a file is modified, its corresponding info file in ns, and any ss that has it also must be modified. It must be made sure that info file modification happens after actual file modification. An example where it may not happen is if the file is present in ss-1 and ss-2, and say some write happens to the file in ss-1. Then the info file is updated in ss-1, and ns and ss-2. Then the ss-1 goes down and ss-2 is left with an unsynced info file and actual file. So make sure info file update happens only after file modification.
- c. In a file, there are **words** and **sentences**. A 'word' is a sequence of ascii characters that are not interrupted by a whitespace. A sentence is a sequence of ascii characters delimited by a '.', '?', '!'. These are the only rules defining words and sentences. For write operations, the file is stored in memory as a linked list of sentences. Each sentence has a linked list of the words inside it.

Operations:

Operations are requested from the client. It sends the operation it wants to the ns. The operation may be of

1. Type-1: client served by the ns
2. Type-2: client served by ns, and ns talks to the ss during the servicing of the operation.
3. Type-3: client served by ss

After every operation, ns provides some feedback to the client about the operation, for example if the operation was successful or not, etc. etc.

NOTE: <filename> in all of the following refers to the names shown in the indexing done by the ns. So if a file is indexed as file.txt, its filename is file.txt. If it is indexed as newdir/file.txt, its filename is newdir/file.txt

Type-1 operations: Client requests and is directly served by ns.

1. VIEW

- a. VIEW: lists all files the user has access to. It checks this access by reading the info file for the corresponding file. If a user has any access to a file (i.e. any combination of r or w or x), it lists that file, along with the access type, like a combination of r, w, x.
- b. VIEW -a: list all files, regardless of if the user who has requested it has access or not.
- c. VIEW -l: list files user has access to, along with the info from info file of that file, like creation time, size, last modified time, and also the access type.
- d. VIEW -al: list all files, regardless of if the user who has requested it has access or not, along with the info from info file of that file, like creation time, size, last modified time, and also the access type.

2. LIST

- a. LIST gives a list of all the users in the system currently, and then a list of all the users that have ever connected.

3. ACCESS

- a. **ADDACCESS -T <filename> <username>**: The owner of the file can give 'T' 'kind of access to a user for a file. If T=R, only read access is given. If T=W, then R and W both are given. If T=X, both R and X are given. After this operation,
- b. **REMACCESS <filename> <username>**: The owner of the file can remove all the access of a particular user from the file.
- c. **REQACCESS -T <filename>**: A user can request 'T' access for a file. This is stored persistently by the ns in a requests list.
- d. **REQLIST**: Shows the requests list.
- e. **APPROVE <index>**: Owner can approve index'th request, and the request is removed from the requests list. If user tries to approve when he is not the owner, an error is given to the user.
- f. **REJECT <index>**: Owner can reject index'th request and the request is removed from the requests list. If user tries to reject when he is not the owner, an error is given to the user.
- g. Note that the above operations change the info file in the ns, so it has to tell each ss with these files to update their info file based on the new info file given by the ns.

4. INFO

- a. **INFO <filename>**: client request file info. Ns checks the info file corresponding to this file and prints all the info in it.

Type-2 operations: Client requests and is served by Ns, and Ns talks to the ss during servicing of this request

1. CREATE

- a. **CREATE <filename>**: client asks to create file. Ns can ask any of the ss to create the file. The file should also be created in the ss backing up the ss in which file was created.

2. DELETE

- a. **DELETE <filename>**: client asks to delete a file. Ns checks if client is the owner of the file. If he is, ns issues command to delete the file in all the ss that the file is in.

3. FOLDER:

- a. **CREATEFOLDER <dirname>**: client asks to create a directory dirname. Ns should create this directory in any of the ss. The directory should be created inside file_dir. The directory should be created in the backup ss of the ss in which the directory was created.
- b. **MOVE <filename> <dirname>**: client asks to move a file from its current location that it can see on the file indexing given by the ns, to the dirname directory. If a directory is not on the same ss as the file, this case should be properly handled.
- c. **VIEWFOLDER <dirname>**: client asks to view all the files inside this directory.

NOTE: Only one level of directory is sufficient. Nested directories need not be handled.

4. CHECKPOINT:

- a. CHECKPOINT <filename> <checkpoint tag>: client asks to checkpoint a filename. Client must have write access to file for this op. All ss with this file must create a checkpoint of this file with the specified checkpoint tag. It is just a copy of the file at that point in time. Checkpoint_dir contains all the checkpoints in a particular ss. If a checkpoint with the same checkpoint tag already exists, it is replaced.
- b. VIEWCHECKPOINT <filename> <checkpoint_tag>: Client asks to view the contents in this particular checkpoint file. Its contents from any one ss that has it should be sent to ns by ss and then ns serves it to client.
- c. REVERT <filename> <checkpoint_tag>: Client asks to revert the file to the state that the checkpoint with this checkpoint tag had saved in. So every instance of the file in each ss is replaced by this operation, with the checkpoint specified.
- d. LISTCHECKPOINTS <filename>: List all the checkpoints associated with this filename on all ss that have this file.

5. UNDO:

- a. UNDO <filename>: Ns checks if the user has write permission for this file. Replaces every instance of filename in every ss that stores it with the past version of the file saved in undo_dir/filename.
- b. Undo file is updated as follows:
 - i. Whenever a new file is created, a corresponding undo file is also created, as specified before.
 - ii. Whenever a write is committed, the current version of the file is saved under undo_dir/filename on all ss that contain the file.
 - iii. Whenever a delete/checkpoint happens, the undo_file is updated
 - iv. Basically with any operation that can update the file state, its corresponding undo file and info file are always updated on every ss that stores the file.

6. EXEC:

- a. EXEC <filename>: Ns checks if the client requesting this operation has execute access on the file. Then the corresponding file is requested by ns. Ns opens this file and executes it LINE-BY-LINE as bash commands. The output is given to the client as the execution happens. So if its like echo hello; some_time_consuming_command; Then hello should be sent to the client and client should not have to wait for completion of some_time_consuming_command to receive output of both echo and some_time_consuming_command together. The project document uses the term “output of the command should be piped to the client” for this.

Type-3 operations: Client requests, ns verifies client request, and tells client to connect to ss. SS then services client directly.

For all of these operations, client will request the ns, ns will verify the client, then tell the ss about the appropriate operation name, and then the ns will make the client connect to the ss. After connecting, ss opens the file appropriately. SS contains in it a list of all files that are

currently in opened. In this, there are appropriate data about the file, for example, file name, number of readers, a global file lock for committing and other places where a global file lock would be needed, and a tokenised version of the file. The tokenisation is done according to the word sentence structure of the file, and it is in memory as a linked list, as described at the starting. Before trying to put a file in memory, the ss checks if the file is already in memory. The ss should be able to serve multiple clients concurrently. So it should have the ability to put multiple files in memory and also be able to allow users to edit /view a particular file concurrently without RACE/deadlock issues.

Write, Stream

1. READ:

- a. READ <filename>: the ns needs to check if client has read permission on this file. The ns then finds one of the ss having this file and makes the client connect to this ss. The ss then sends the required content of the file to the client.

2. WRITE:

- a. WRITE <filename> <sentence_number>:
 - i. The ns checks if the user has write permissions on this file. Then it asks the ss to lock the sentence with index sentence_number. The ns connects the client to this ss. The file to be written to is put in memory, if it isn't already. Then the sentence required is locked. So there needs to be a per sentence lock. (NOTE: THE client retains the sentence that he has locked. If the file has changed, i.e. some sentences have been added before this client's sentence, then the index of the sentence changes. Dont worry about this. The client retains access to the sentence he had locked initially. No one else can write to this sentence while this client is writing to it, however other sentences can be written to and updated.) Then the sentence is given to the client. Now the client has that sentence. The client then writes to the sentence in the following way:
<word_index> <content>
<word_index> <content>
Note that the word index keeps updating after every such line. So subsequent <word_index> stuff should be handled properly. A naive method of traversing the linked list every single time is sufficient.
 - ii. NOTE: word_index and sentence_index are 0-indexed. If they go out of bounds, they should give an out-of-bounds error. However, suppose there are 4 sentences in a file and a client requests to write to the sentence with index 4. In this case the sentence index is indeed out of bounds. However, since it is just out of bounds, it is considered that the user is trying to append to the file. So a new sentence node is created in the linked list. And then this sentence node is locked and given to the user to modify. Note that during the addition of this new sentence node to the linked list, the entire list should be held by a global lock since it is changing. In general, whenever the entire sentence linked list is being edited, for example when this kind of sentence appending happens or

when a write is committed, the global lock must be held. While the global lock is held, no write requests/read/stream requests are allowed on the file. The global lock is given up as soon as the commit is done or the sentence has been added. Then the lock of the sentence is activated and the sentence is given to the user to edit. NOTE that multiple people can edit the file simultaneously. So they can edit multiple sentences at once.

- iii. A user signals that a write is done by typing ETIRW. This sends the new things that the user has written to the ss and starts the commit process, which is started by locking the whole file structure. Note that the user can only add to the current structure of the file, the user cannot remove any characters. Also note that during the write, the user may add multiple sentences. Let S1 be the sentence the user initially locked. After the user is done writing, he sends to the ss possibly a bunch of sentences. These need to be placed appropriately. Let s0 be the sentence in the linked list right before the sentence that the user has locked, and let s2 be the sentence right after the sentence that user has locked (note that this linked list is the one after the whole file has been locked.) The first sentence that the user has given back to the ss should be right after s0, and the last sentence should be right before s2.
After this, the file is actually written to. Note that this write must be atomic. S write the tokenised stuff into a temp file, save the old file to undo_dir, and then delete the old file. Now rename the temp file to the actual name of the file. Update the undo file corresponding to this file. Then release the global commit lock. Note that when the commit lock is held, the file cannot be accessed by anyone or anything except the thing that locked it. Not even the undo/checkpoint commands or something can access the file.

3. STREAM

- a. STREAM <filename>: The ns checks if the client has permission to access the file. Then ns connects the client to the ss. The ss starts streaming the contents of the file to the client, word-by-word, every 0.1 seconds. For this purpose, ss copies the file into some temp file, tokenises it and streams this tokens word-by-word. This temp file is streamed across. The temp file is not a part of the files list, and does not contribute to a file being opened. STREAM is blocked when the global file lock is active.
- b. The stream ends when the file has been fully streamed by the ss. The ss sends a STOP packet to the ns. The ss terminates this client connection, or asks the ns to terminate this connection.
- c. In case the server crashes during stream, the ns should detect this and the client should be appropriately notified.

Backup Plan:

1. In the event that a storage server goes down, there must be some concept of backups.

2. Every time a ss connects, the ns asks if it is empty or not. If it is not, then it cannot be used as a backup server. If it is empty, ns checks if there are any currently present ss that do not have a backup, and are not backups themselves. If this is the case, it makes the new ss a backup of the previous ss that did not have a backup.
3. The ss copies all its files into the backup. All the directories, info_dir, file_dir, undo_dir, checkpoint_dir are copied. For every file in backup, the info about the backup ss is added to the files indexing done by the ns. So each file which is in a server that has been backed up, will have info of atleast two of the ss, one actual ss, and one backup.
4. After any change to the file, the ss sends the changes to the backup server and changes are copied by the backup server.
5. In the event that an ss goes down, its details should be removed from the file indexing. The backup will be used to serve these requests instead. Now when the actual server comes back up, the backup server can sync with the ss that had gone down to reflect any changes.
6. If either the ss or the backup are down, and the other is up, and another empty ss comes up, then that empty ss should be considered as the backup server for this ss/backup ss that's still alive. Or any ss.
7. Backup have to be persistent. That means if ss-2 becomes a backup for ss-1, then they are bound together forever. SS need not be boolean: either ss or backup. An ss can be a backup ss, as well as being backed up by another backup ss. All of these backups report to the same overall ss.

Logging and informing:

1. The ns must log all its activities, any packet it has sent, any acknowledgement it has received, any client/server that has connected to it etc. etc.
2. After every operation, ns must give some feedback to the user about the operation.
3. An ss must also log its activities.

Error handling:

1. Defined error codes must exist. For example "file not found", "ss not found", etc. etc. a bunch more stuff wherever appropriate.
2. Good error messages must be printed.